

PLSQL_EXPORT_EXCEL

Pure PL/SQL XLSX generator for Oracle 12c / 19c.

No APEX · No UTL_FILE · No shell scripts · No external tools · No client installs required.

Overview

This project solves a common problem in locked-down client VDI environments where you cannot install software, APEX is unavailable, yet you need to export SQL query results as a multi-sheet Excel workbook.

The solution is two PL/SQL packages that generate a valid `.xlsx` file entirely in memory as a `BLOB`, stored in a database table and downloadable directly from SQL Developer or PL/SQL Developer.

Packages

Package	Role
PKG_XLSX_EXPORT	Core engine — ZIP builder, XLSX XML generator, BLOB output
PKG_XLSX_WRAPPER	Smart wrapper — auto-naming, UID tracking, notifications, purge
PKG_XLSX_DIRECT	SQL-callable function — returns BLOB directly, no table required

How it works

Your SQL Session

```
└─ PKG_XLSX_WRAPPER.generate_xlsx(workbook_name, sheet_sql_1, sheet_sql_2, ...)
    │
    │ └─ Resolves workbook name (supplied or auto: SCHEMA_YYYYMMDD_HH24MISS)
    │ └─ Resolves worksheet names (supplied or auto-derived from SQL table name)
    │ └─ Calls PKG_XLSX_EXPORT internally
    │     │ └─ DBMS_SQL → executes each query, captures columns + rows
    │     │ └─ XML builder → generates XLSX parts (workbook, sheets, styles, sh
    │     │ └─ ZIP builder → assembles raw ZIP bytes as BLOB (pure PL/SQL, no U
    │ └─ Stamps a unique UID on the saved record
    └─ Prints success notification + ready-to-run download SQL
```

- └ Auto-purges exports older than 7 days
- └ `PKG_XLSX_DIRECT.generate_xlsx(workbook_name, queries)` ← SQL-callable
 - ├ No table access required – BLOB returned directly to the caller
 - ├ Queries passed as a single delimited string (separator: \$)
 - ├ Sheet names auto-derived or supplied as a second delimited string
 - └ Callable from `SELECT ... FROM DUAL` in read-only environments

Prerequisites

Requirement	Notes
Oracle 12c or 19c	Tested on both
<code>CREATE PACKAGE</code> privilege	On your schema
<code>CREATE TABLE</code> privilege	For output and audit tables
<code>DBMS_SQL</code> execute privilege	Usually granted by default
<code>DBMS_LOB</code> execute privilege	Usually granted by default
SQL Developer or PL/SQL Developer	For downloading the BLOB as <code>.xlsx</code>

Verify your privileges before deploying:

```
SELECT PRIVILEGE FROM USER_SYS_PRIVS
WHERE PRIVILEGE IN ('CREATE PACKAGE','CREATE TABLE','CREATE PROCEDURE')
ORDER BY 1;
```

Installation

Deploy in this exact order. Each file depends on the previous one.

Deployment matrix — what to deploy per environment

Environment	PKG_XLSX_EXPORT	PKG_XLSX_WRAPPER	PKG_XLSX_DIRECT
Full privileges			

(CREATE TABLE etc.)	✅ Required	✅ Deploy	❑ Optional
Read-only (no CREATE TABLE)	✅ Required	❌ Skip	✅ Deploy

Step 1 — Deploy the core engine (ALL environments)

`PKG_XLSX_EXPORT` has **zero table references**. It compiles cleanly in any schema regardless of privileges — read-only or not.

```
@PKG_XLSX_EXPORT.sql
```

What it creates:

Object	Type	Purpose
<code>PKG_XLSX_EXPORT</code>	Package spec	Public API: <code>init</code> , <code>add_sheet</code> , <code>build_blob</code> , <code>get_workbook_name</code>
<code>PKG_XLSX_EXPORT</code>	Package body	ZIP builder + XLSX engine. No table access.

Verify:

```
SELECT OBJECT_NAME, OBJECT_TYPE, STATUS
FROM   USER_OBJECTS
WHERE  OBJECT_NAME = 'PKG_XLSX_EXPORT';
```

Status must be `VALID`. If not:

```
SELECT LINE, TEXT FROM USER_ERRORS WHERE NAME = 'PKG_XLSX_EXPORT' ORDER BY LINE;
```

Step 2a — Full-privilege environment: deploy the wrapper

`PKG_XLSX_WRAPPER` **owns the table**. It creates `XLSX_EXPORT_RESULTS` and `XLSX_EXPORT_LOG`, then calls `PKG_XLSX_EXPORT.build_blob` and does its own INSERT. Requires `CREATE TABLE` privilege.

```
@PKG_XLSX_WRAPPER.sql
```

What it creates:

Object	Type	Purpose
XLSX_EXPORT_RESULTS	Table	Stores generated XLSX BLOBs with UID
UX_XLSX_EXPORT_UID	Unique index	Enforces UID uniqueness
XLSX_EXPORT_LOG	Table	Audit trail of all actions
PKG_XLSX_WRAPPER	Package spec + body	Auto-naming, UID, purge, notifications

Verify:

```
SELECT OBJECT_NAME, OBJECT_TYPE, STATUS
FROM   USER_OBJECTS
WHERE  OBJECT_NAME = 'PKG_XLSX_WRAPPER';
```

Step 2b — Read-only environment: deploy the direct function

`PKG_XLSX_DIRECT` requires **only** `CREATE PACKAGE`. It calls `PKG_XLSX_EXPORT.build_blob` and returns the BLOB straight to the SQL caller — no INSERT, no table, nothing.

```
@PKG_XLSX_DIRECT.sql
```

What it creates:

Object	Type	Purpose
PKG_XLSX_DIRECT	Package spec + body	SQL-callable BLOB function, no table access

Verify:

```
SELECT OBJECT_NAME, OBJECT_TYPE, STATUS
FROM   USER_OBJECTS
WHERE  OBJECT_NAME = 'PKG_XLSX_DIRECT';
```

Always use `PKG_XLSX_WRAPPER` for day-to-day exports. Use `PKG_XLSX_EXPORT` directly only if you need programmatic BLOB access.

Naming behaviour

Workbook name (the `.xlsx` filename):

Scenario	Result
<code>p_workbook_name</code> supplied	That name is used
<code>p_workbook_name</code> not supplied	<code>SCHEMA_YYYYMMDD_HH24MISS</code>
Name already exists in table	Auto-suffixed: <code>MY_REPORT_01</code> , <code>MY_REPORT_02</code> , ...

Worksheet name (each tab inside the workbook):

Scenario	Result
<code>p_sheetN_name</code> supplied	That name is used
Not supplied — single table in SQL	Table name (e.g. <code>EMPLOYEES</code>)
Not supplied — multiple tables/JOINS	<code>COMPLEX_01</code> , <code>COMPLEX_02</code> , ...
Not supplied — unparseable SQL	<code>SHEET_1</code> , <code>SHEET_2</code> , ...
Duplicate name within workbook	Auto-suffixed: <code>EMPLOYEES_01</code>

Example 1 — No names supplied (fully automatic)

```
DECLARE
  v_uid VARCHAR2(32);
BEGIN
  v_uid := PKG_XLSX_WRAPPER.generate_xlsx(
    p_sheet1_sql => 'SELECT OBJECT_NAME, OBJECT_TYPE, STATUS, CREATED
                    FROM USER_OBJECTS ORDER BY OBJECT_TYPE',
    p_sheet2_sql => 'SELECT S.SEGMENT_NAME, S.BYTES, E.EXTENT_ID
                    FROM USER_SEGMENTS S
                    JOIN  USER_EXTENTS E ON E.SEGMENT_NAME = S.SEGMENT_NAME
                    WHERE ROWNUM <= 100',
```

```

        p_sheet3_sql => 'SELECT NAME, TYPE, LINE, TEXT
                        FROM USER_SOURCE WHERE ROWNUM <= 200'

    );
    DBMS_OUTPUT.PUT_LINE('UID: ' || v_uid);
END;
/
-- Workbook → MYSCHEMA_20240522_143055
-- Sheet 1 → USER_OBJECTS
-- Sheet 2 → COMPLEX_01 (two tables joined)
-- Sheet 3 → USER_SOURCE

```

Example 2 — Workbook name supplied, sheet names auto-derived

```

DECLARE
    v_uid VARCHAR2(32);
BEGIN
    v_uid := PKG_XLSX_WRAPPER.generate_xlsx(
        p_workbook_name => 'HR_MONTHLY_REPORT',
        p_sheet1_sql     => 'SELECT EMPLOYEE_ID, FIRST_NAME, LAST_NAME, SALARY
                        FROM EMPLOYEES ORDER BY LAST_NAME',
        p_sheet2_sql     => q'[SELECT E.FIRST_NAME||' '||E.LAST_NAME AS NAME,
                        D.DEPARTMENT_NAME, E.SALARY
                        FROM EMPLOYEES E
                        JOIN DEPARTMENTS D
                        ON D.DEPARTMENT_ID = E.DEPARTMENT_ID
                        ORDER BY E.SALARY DESC]'
    );
END;
/
-- Workbook → HR_MONTHLY_REPORT (or HR_MONTHLY_REPORT_01 if name exists)
-- Sheet 1 → EMPLOYEES
-- Sheet 2 → COMPLEX_01

```

Example 3 — All names explicitly provided

```

DECLARE
    v_uid VARCHAR2(32);
BEGIN
    v_uid := PKG_XLSX_WRAPPER.generate_xlsx(
        p_workbook_name => 'AUDIT_PACK',
        p_sheet1_name   => 'Access Log',

```

```

p_sheet1_sql      => 'SELECT * FROM ACCESS_LOG      WHERE ROWNUM <= 500',
p_sheet2_name     => 'Error Summary',
p_sheet2_sql      => 'SELECT * FROM ERROR_SUMMARY WHERE ROWNUM <= 500',
p_sheet3_name     => 'User Activity',
p_sheet3_sql      => 'SELECT * FROM USER_ACTIVITY WHERE ROWNUM <= 500'
);
END;
/

```

Example 4 — WHERE clause with a character parameter

When your SQL contains a string literal (e.g. `LIKE 'A%'`), the single quotes must be escaped because the SQL itself is passed as a PL/SQL string.

Rule: every single quote inside a regular PL/SQL string must be doubled (`'` → `''`).

```

-- Plain SQL you would type at a prompt:
--   WHERE ENAME LIKE 'A%'
--
-- Inside a PL/SQL string, each ' becomes '':
--   WHERE ENAME LIKE ''A%' '

```

```

DECLARE
  v_uid VARCHAR2(32);
BEGIN
  v_uid := PKG_XLSX_WRAPPER.generate_xlsx(
    p_workbook_name => 'FILTERED_EMPLOYEES',
    p_sheet1_name   => 'EMP_NAME_A',
    p_sheet1_sql    =>
      'SELECT EMPNO, ENAME, JOB, SAL, HIREDATE ' ||
      'FROM   EMP ' ||
      'WHERE  ENAME LIKE ''A%' ' ||
      'ORDER BY ENAME'
  );
END;
/

```

Alternative — use Oracle's `q' [...]'` quoting mechanism (recommended for readability):

```

DECLARE
  v_uid VARCHAR2(32);
BEGIN
  v_uid := PKG_XLSX_WRAPPER.generate_xlsx(
    p_workbook_name => 'FILTERED_EMPLOYEES',

```

```

p_sheet1_name    => 'EMP_NAME_A',
p_sheet1_sql     => q'[
    SELECT EMPNO, ENAME, JOB, SAL, HIREDATE
    FROM   EMP
    WHERE  ENAME LIKE 'A%'
    ORDER  BY ENAME
]'
);
END;
/

```

Inside `q'[...]` the bracket pair acts as the string delimiter — you write single quotes naturally without doubling them.

Example 5 — WHERE clause where the value itself contains a single quote

Scenario: filter by `O'BRIEN` — a value with an embedded apostrophe.

There are three techniques. **Technique B or C are recommended** to avoid confusion.

Technique A — doubled-quote escaping (regular string)

Every quote in the entire PL/SQL string is doubled, including the embedded apostrophe:

```

-- Plain SQL target:  WHERE ENAME = 'O'BRIEN'
-- In PL/SQL string:  ''O''''BRIEN''
--
-- Breakdown of  ''O''''BRIEN''  :
--  ''      → opening  '  of the SQL literal
--  O       → the letter O
--  ''''    → the embedded apostrophe (close string + reopen string)
--  BRIEN   → rest of the name
--  ''      → closing  '  of the SQL literal

DECLARE
    v_uid VARCHAR2(32);
BEGIN
    v_uid := PKG_XLSX_WRAPPER.generate_xlsx(
        p_workbook_name => 'APOSTROPHE_DEMO',
        p_sheet1_name    => 'EMP_OBRIEN',
        p_sheet1_sql     =>
            'SELECT EMPNO, ENAME, JOB, SAL ' ||
            'FROM   EMP ' ||
            'WHERE  ENAME = ''O''''BRIEN'' ' ||
            'ORDER  BY EMPNO'
    );

```



```

);
END;
/

```

Technique B — `q'[...]` alternative quoting (recommended)

The PL/SQL wrapper quotes vanish. Only the SQL-level embedded apostrophe still needs doubling (that is SQL syntax, not PL/SQL):

```

DECLARE
    v_uid VARCHAR2(32);
BEGIN
    v_uid := PKG_XLSX_WRAPPER.generate_xlsx(
        p_workbook_name => 'APOSTROPHE_DEMO',
        p_sheet1_name   => 'EMP_OBRIEN',
        p_sheet1_sql    => q'[
            SELECT EMPNO, ENAME, JOB, SAL
            FROM   EMP
            WHERE  ENAME = 'O''BRIEN'
            ORDER  BY EMPNO
        ]'
    );
END;
/

```

Technique C — build SQL dynamically using `CHR(39)` (cleanest for parameterised values)

Avoids all escaping confusion. Use this when the filter value comes from a variable:

```

DECLARE
    v_uid  VARCHAR2(32);
    v_name VARCHAR2(100) := 'O''BRIEN'; -- PL/SQL variable holds the actual value
    v_sql  CLOB;
BEGIN
    -- CHR(39) is the single-quote character
    v_sql := 'SELECT EMPNO, ENAME, JOB, SAL '
        || 'FROM   EMP '
        || 'WHERE  ENAME = ' || CHR(39) || v_name || CHR(39) || ' '
        || 'ORDER  BY EMPNO';

    v_uid := PKG_XLSX_WRAPPER.generate_xlsx(
        p_workbook_name => 'APOSTROPHE_DEMO',
        p_sheet1_name   => 'EMP_OBRIEN',
        p_sheet1_sql    => v_sql
    );
END;
/

```

```
END;  
/
```

Single-quote escaping cheat sheet

SQL you want	Regular PL/SQL string	q'[...]'
WHERE COL = 'SMITH'	'SMITH'	'SMITH'
WHERE COL LIKE 'A%'	'A%'	'A%'
WHERE COL = 'O''BRIEN'	'O''''BRIEN''	'O''BRIEN'
WHERE COL = 'ST JOHN''S'	'ST JOHN''''S''	'ST JOHN''S'
'MR ' ENAME	'MR '	'MR '

Key rule: in a regular PL/SQL string every single quote is doubled. In q'[...]' only the SQL-level embedded apostrophes are doubled — the PL/SQL wrapper quotes disappear entirely.

Download the generated file

When generate_xlsx completes, DBMS_OUTPUT prints this automatically:

```
=====
EXPORT COMPLETE
-----

Workbook   : HR_MONTHLY_REPORT.xlsx
Export ID  : 5
UID        : A3F29C1B...
Size       : 48.3 KB
-----

DOWNLOAD — run in SQL Developer / PL/SQL Developer:

SELECT XLSX_BLOB
FROM   XLSX_EXPORT_RESULTS
WHERE  EXPORT_UID = 'A3F29C1B...';

Save the BLOB cell as: HR_MONTHLY_REPORT.xlsx
-----
```

DELETE when done:

```
EXEC PKG_XLSX_WRAPPER.delete_export('A3F29C1B...');  
=====
```

In SQL Developer: run the SELECT above → click the BLOB cell → pencil/download icon → Save As → rename to `.xlsx`

In PL/SQL Developer: run the SELECT above → right-click the BLOB cell → Save to File → rename to `.xlsx`

List all exports

```
EXEC PKG_XLSX_WRAPPER.list_exports;
```

Re-print download SQL for a known UID

```
EXEC PKG_XLSX_WRAPPER.print_retrieval_sql('A3F29C1B...');
```

Delete a specific export

```
EXEC PKG_XLSX_WRAPPER.delete_export('A3F29C1B...');
```

Purge old exports manually

Exports older than 7 days are purged automatically on every `generate_xlsx` call. To purge manually:

```
-- Default: remove exports older than 7 days  
EXEC PKG_XLSX_WRAPPER.purge_old_exports;  
  
-- Custom retention period (e.g. 3 days)  
EXEC PKG_XLSX_WRAPPER.purge_old_exports(3);
```

View audit log

Every action (INITIATED, COMPLETED, FAILED, DELETED, PURGED) is recorded:

```
SELECT ACTION, MESSAGE, LOG_TIME
FROM   XLSX_EXPORT_LOG
ORDER  BY LOG_TIME DESC;
```

Direct BLOB access (advanced / programmatic)

If you need the BLOB directly without storing it (e.g. to pipe to `UTL_MAIL`):

```
DECLARE
    v_blob BLOB;
BEGIN
    PKG_XLSX_EXPORT.init('My_Export');
    PKG_XLSX_EXPORT.add_sheet('Data', 'SELECT * FROM MY_TABLE WHERE ROWNUM <= 1000'
    v_blob := PKG_XLSX_EXPORT.build_blob;
    -- use v_blob as needed
    DBMS_OUTPUT.PUT_LINE('Size: ' || DBMS_LOB.GETLENGTH(v_blob) || ' bytes');
END;
/
```

PKG_XLSX_DIRECT — Read-only environment usage

Use this package when you cannot create tables. All queries are passed as a **single delimited string** separated by `§` (the section sign), making it callable directly from a `SELECT` statement.

How the delimiter works

```
-- Single query — no delimiter needed
'SELECT * FROM EMP'

-- Multiple queries — separate with PKG_XLSX_DIRECT.DELIM (§)
'SELECT * FROM EMP'
|| PKG_XLSX_DIRECT.DELIM
|| 'SELECT * FROM DEPT'
|| PKG_XLSX_DIRECT.DELIM
```

```
|| 'SELECT * FROM SALGRADE'
```

Overload 1 — sheet names auto-derived

```
SELECT PKG_XLSX_DIRECT.generate_xlsx(  
    p_workbook_name => 'Deposit_Issue',  
    p_queries       => 'SELECT * FROM ICTM_ACC WHERE ROWNUM <= 10'  
    || PKG_XLSX_DIRECT.DELIM  
    || q'[SELECT * FROM STTM_CUST_ACCOUNT  
        WHERE CUST_NAME LIKE 'A%']'  
)  
FROM DUAL;  
-- Sheet 1 → ICTM_ACC  
-- Sheet 2 → STTM_CUST_ACCOUNT
```

Overload 2 — sheet names explicitly provided

```
SELECT PKG_XLSX_DIRECT.generate_xlsx(  
    p_workbook_name => 'Deposit_Issue',  
    p_sheet_names   => 'Accounts' || PKG_XLSX_DIRECT.DELIM || 'Customers',  
    p_queries       => 'SELECT * FROM ICTM_ACC WHERE ROWNUM <= 10'  
    || PKG_XLSX_DIRECT.DELIM  
    || q'[SELECT * FROM STTM_CUST_ACCOUNT  
        WHERE CUST_NAME LIKE 'A%']'  
)  
FROM DUAL;  
-- Sheet 1 → Accounts  
-- Sheet 2 → Customers
```

No workbook name — auto-named

```
SELECT PKG_XLSX_DIRECT.generate_xlsx(  
    p_queries => 'SELECT EMPNO, ENAME, SAL FROM EMP ORDER BY ENAME'  
)  
FROM DUAL;  
-- Workbook → MYSCHEMA_20240522_143055  
-- Sheet 1 → EMP
```

WHERE clause — character filter (`LIKE 'A%'`)

```
-- Using q'[...]' (recommended — no escaping needed):  
SELECT PKG_XLSX_DIRECT.generate_xlsx(  
    p_queries => 'SELECT * FROM EMP WHERE ENAME LIKE 'A%'  
)
```

```

        'EMP_Report',
        q'[SELECT EMPNO, ENAME, JOB, SAL FROM EMP WHERE ENAME LIKE 'A%']'
    )
FROM DUAL;

-- Using regular string (double every quote):
SELECT PKG_XLSX_DIRECT.generate_excel(
    'EMP_Report',
    'SELECT EMPNO, ENAME, JOB, SAL FROM EMP WHERE ENAME LIKE ''A%'''
)
FROM DUAL;

```

WHERE clause — value contains an apostrophe (O'BRIEN)

```

-- q'[...]' — only the SQL-level apostrophe needs doubling:
SELECT PKG_XLSX_DIRECT.generate_excel(
    'EMP_Report',
    q'[SELECT EMPNO, ENAME, JOB FROM EMP WHERE ENAME = 'O''BRIEN']'
)
FROM DUAL;

-- Regular string — every quote doubled:
SELECT PKG_XLSX_DIRECT.generate_excel(
    'EMP_Report',
    'SELECT EMPNO, ENAME, JOB FROM EMP WHERE ENAME = ''O''''BRIEN'''
)
FROM DUAL;

```

How to download from SQL Developer / PL/SQL Developer

1. Run any `SELECT PKG_XLSX_DIRECT.generate_excel(...) FROM DUAL` query
2. The result grid shows one row with a BLOB column
3. **SQL Developer** — click the BLOB cell → pencil/download icon → Save As → rename to `Workbook.xlsx`
4. **PL/SQL Developer** — right-click the BLOB cell → Save to File → rename to `Workbook.xlsx`
5. Open in Excel

Changing the delimiter

The default delimiter `§` covers most cases. If your SQL text contains `§`, change the `DELIM`

constant in the package body and recompile:

```
-- In PKG_XLSX_DIRECT body Section A, change:  
DELIM CONSTANT VARCHAR2(3) := '$';  
-- to any string your SQL won't contain, e.g.:  
DELIM CONSTANT VARCHAR2(3) := '|~|';
```

API Reference

PKG_XLSX_DIRECT (read-only environments)

Function	Parameters	Description
<code>generate_xlsx</code> (overload 1)	<code>p_workbook_name</code> , <code>p_queries</code>	Queries delimited by <code>\$</code> . Sheet names auto-derived. Returns BLOB.
<code>generate_xlsx</code> (overload 2)	<code>p_workbook_name</code> , <code>p_sheet_names</code> , <code>p_queries</code>	Same, plus explicit sheet names delimited by <code>\$</code> . Returns BLOB.
<code>DELIM</code>	constant	The query/name separator character (<code>\$</code> by default).

PKG_XLSX_WRAPPER

Procedure / Function	Parameters	Description
<code>generate_xlsx</code>	<code>p_workbook_name</code> , <code>p_sheet1_name</code> , <code>p_sheet1_sql</code> , ... up to sheet 10	Builds workbook, saves BLOB, returns UID
<code>delete_export</code>	<code>p_export_uid</code>	Deletes one export by UID
<code>purge_old_exports</code>	<code>p_days</code> (default 7)	Deletes exports older than N days
<code>list_exports</code>	—	Prints all exports to DBMS_OUTPUT
		Re-prints download

<code>print_retrieval_sql</code>	<code>p_export_uid</code>	SQL for a UID
----------------------------------	---------------------------	---------------

PKG_XLSX_EXPORT (core engine)

Procedure / Function	Description
<code>init(p_workbook_name)</code>	Reset state, set workbook name
<code>add_sheet(p_sheet_name, p_sql)</code>	Register a worksheet (max 31-char name)
<code>build</code>	Execute queries, build XLSX, persist to table. Returns <code>EXPORT_ID</code>
<code>build_blob</code>	Same as <code>build</code> but returns raw BLOB without persisting
<code>get_blob(p_export_id)</code>	Retrieve a saved BLOB by numeric ID

Supported Oracle Data Types

Oracle Type	Excel Output	Notes
<code>VARCHAR2</code> , <code>CHAR</code> , <code>NVARCHAR2</code>	String	Via shared strings table
<code>NUMBER</code> , <code>BINARY_FLOAT</code> , <code>BINARY_DOUBLE</code>	Numeric	Native Excel number cell
<code>DATE</code> , <code>TIMESTAMP</code> (all variants)	String	Formatted <code>YYYY-MM-DD HH24:MI:SS</code>
<code>CLOB</code> , <code>LONG</code>	String	Truncated to 255 chars per cell
All others	String	Safe fallback

XLSX Output Characteristics

- **Header row** — Bold, blue background (#4472C4), centred, auto-generated from column names

- **Encoding** — UTF-8 throughout
- **Compression** — STORE mode (no deflate); Excel opens it natively, no corruption risk
- **Shared strings** — All text centralised in `xl/sharedStrings.xml` per OOXML spec
- **Multi-sheet** — Up to 10 sheets per call via wrapper; unlimited via base package

Limitations

Limitation	Detail
No cell formulas	Data-only export
No column auto-width	All columns use Excel default width
CLOB/LONG truncated at 255 chars	Per-cell practical limit
No bind variables in SQL	Embed literal values before passing to <code>add_sheet</code>
No DEFLATE compression	STORE mode ZIP — larger files for text-heavy data
Max sheet name 31 chars	Excel hard limit, enforced by both packages
Max 10 sheets via wrapper	Use <code>PKG_XLSX_EXPORT</code> directly for more

Compatibility

Environment	Status
Oracle 12c	✓ Tested
Oracle 19c	✓ Tested
SQL Developer (BLOB download)	✓ Tested
PL/SQL Developer (BLOB download)	✓ Tested
Microsoft Excel 2016+	✓ Tested
LibreOffice Calc	✓ Tested

License

MIT — free to use, modify, and distribute.